Министерство науки и высшего образования РФ

АРЗАМАССКИЙ ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ (ФИЛИАЛ)
ФЕДЕРАЛЬНОГО ГОСУДАРСТВЕННОГО БЮДЖЕТНОГО
ОБРАЗОВАТЕЛЬНОГО УЧРЕЖДЕНИЯ ВЫСШЕГО ОБРАЗОВАНИЯ «НИЖЕГОРОДСКИЙ
ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ ИМ. Р.Е.АЛЕКСЕЕВА»
(АПИ НГТУ)

Лабораторная работа № 5
по дисциплине «Логика и архитектура вычислительных сред»
по проекту «RL-агент для игры в клеточный Dungeon Crawler»

Выполнил:
студент гр.МА 26 ПМ
Варганова Э.Р.

Проверил:
Комаров А.О.

Арзамас, 2026

1. Цель и проблема проекта

Цель: создать программную среду игры-подземелья (cell-based dungeon crawler) и обучить RL-агента, который умеет: исследовать карту, собирать награды (монеты), избегать ловушек и врагов.

Проблема: правило-ориентированный бот (hand-crafted FSM) достигает «потолка» качества: он хорош там, где заложил поведение разработчик, но не адаптируется к новой геометрии уровня, изменению числа врагов и расстановке ловушек. Задача проекта — показать, что PPO-агент с CNN-политикой обучается на пиксельном представлении карты и превосходит бейзлайны, а также выстроить вокруг обучения production-подобный инфраструктурный контур: телеметрия через Kafka, трекинг экспериментов в MLflow, inference через HTTP API.

Ключевые вопросы, на которые отвечает проект:

- обучается ли агент на наблюдении [6, H, W] без знания внутренней логики симулятора;
- насколько итоговая средняя награда выше, чем у random- и FSM-агентов;
- как разделить систему на слабосвязанные сервисы (симуляция, обучение, инференс, телеметрия);
- как обеспечить воспроизводимость эпизодов и их полный «проигрыш» по журналу событий.

2. Требования и ограничения

2.1. Функциональные требования

- Генерация уровня: комнаты + коридоры, детерминированная по seed, воспроизводимая.

- Среда совместима с Gymnasium API: reset(), step(), observation_space, action_space.
- Наблюдение агента — тензор [6, H, W] float32 (6 каналов: стены, игрок, враги, монеты, ловушки, зоны опасности).
- Три бейзлайна: random-агент, FSM-агент («robot»), обучаемый PPO-агент.
- Каждое событие симуляции публикуется в Kafka-топик; consumer агрегирует сводку.
- Трекинг экспериментов: reward, episode length, win rate, entropy — в MLflow.
- Inference service: HTTP API предсказания действия по наблюдению.

2.2. Нефункциональные требования

- Воспроизводимость: фиксация seed, версий библиотек, гиперпараметров.
- Отказоустойчивость телеметрии: при недоступности Kafka producer деградирует в no-op, обучение не блокируется.
- Тестируемость: юнит- и интеграционные тесты (pytest), включая replay-тест Event Sourcing.

2.3. Ограничения и допущения

- Полная наблюдаемость: агент видит всю карту (не partial observability).
- Дискретное пространство действий: 4 направления (up/down/left/right), без боевых действий.
- Детерминированная механика врагов: стратегии random / patrol / chase; враги не «умнеют».
- Обучение офлайн в среде-симуляторе; против реальных игроков сервис не применяется.

- Kafka/MLflow — необязательная зависимость для локального запуска: без них пайплайн работает.

3. Описание входных и выходных данных

3.1. Входные данные

Сущность	Формат	Описание
seed	Int	Сид генерации уровня и стохастических стратегий
H×W	Int	Размер сетки уровня (по умолчанию 16×24)
hyperparameters PPO	CLI/JSON	n_envs, total_timesteps, learning_rate, gamma, ent_coef
observation	tensor [6,H,W] float32	Каналы: 0 стены, 1 игрок, 2 враги, 3 монеты, 4 ловушки, 5 danger-зоны
HTTP /predict	JSON {"obs": [[...]]}	Наблюдение для inference service

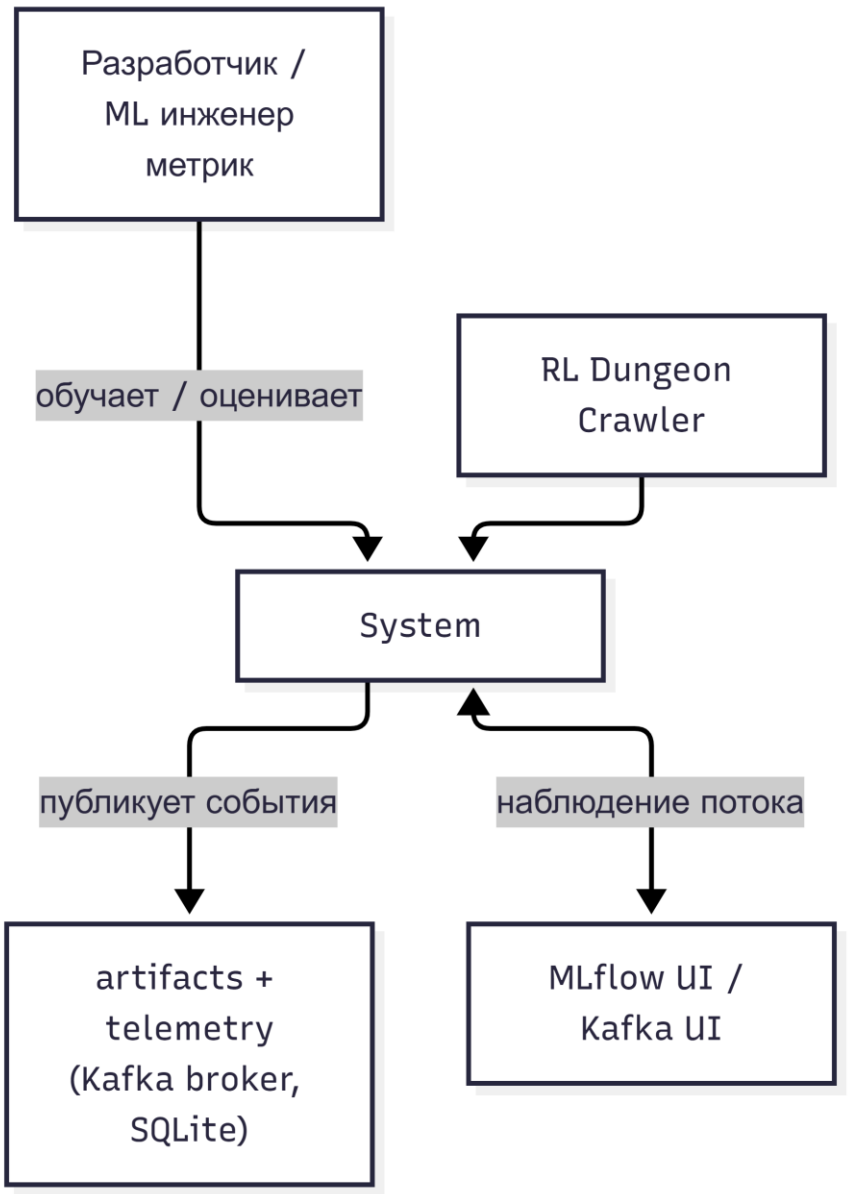
3.2. Выходные данные

Сущность	Формат	Описание
action	int ∈ {0,1,2,3}	Дискретное действие (движение)
reward	Float	Мгновенная награда шага
info (per step)	Dict	agent_state, event_type, done_reason и др.
event message	JSON в Kafka	см. раздел 6
model	SB3 .zip	Веса политики + среды

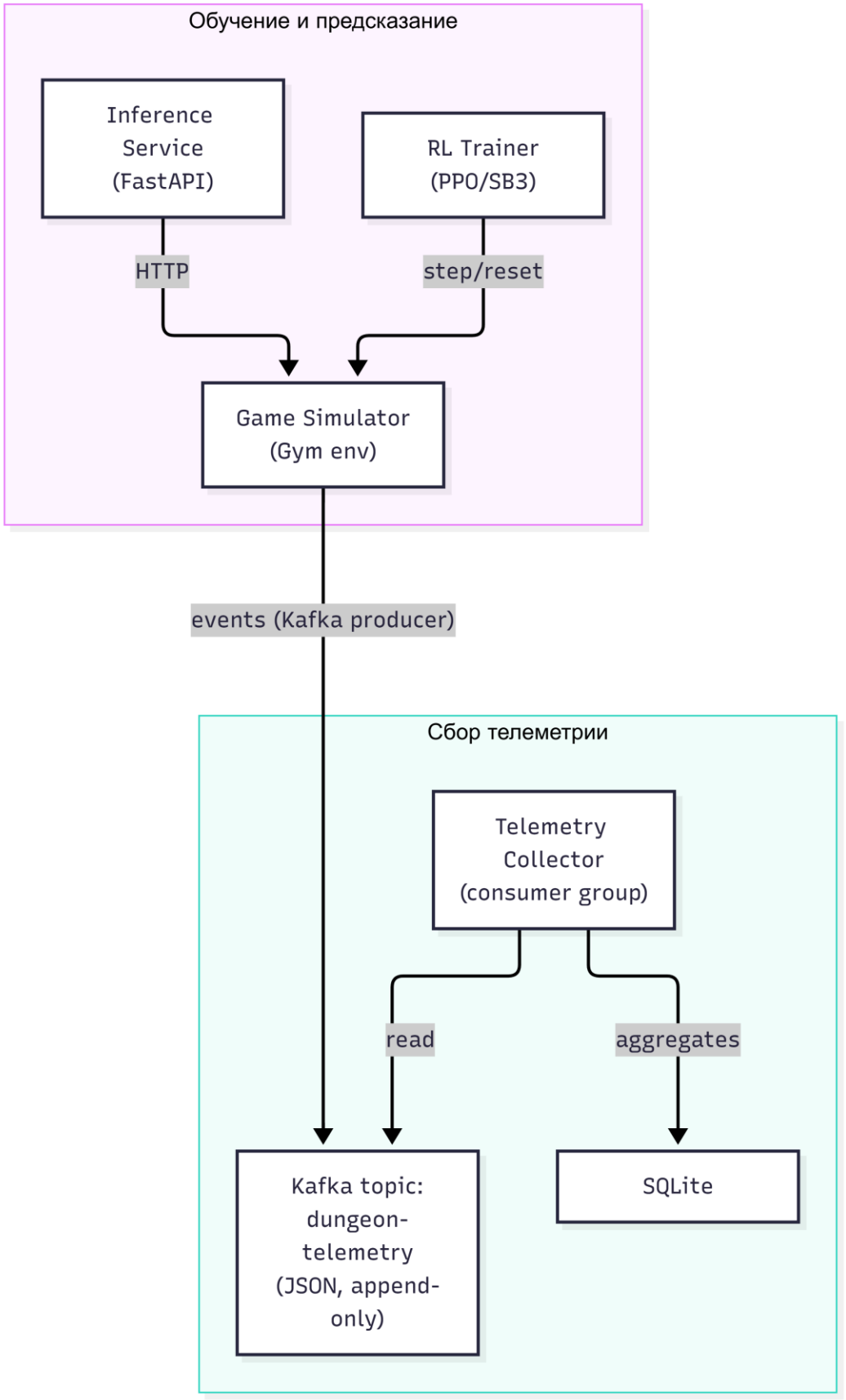
checkpoint		
MLflow run	metrics/params/artifacts	Кривые обучения, конфиг запуска
analytics	SQLite (aggregate)	Сводки эпизодов для анализа

4. Архитектурная схема (C4)

4.1. C4 Level 1 — System Context



4.2. C4 Level 2 — Containers



Эксперименты: `RL Trainer → MLflow server (SQLite backend, artifacts dir)`

Контейнеры слабо связаны через контракты: `Gym API (trainer ↔ simulator)`, `HTTP (inference ↔ simulator)`, `Kafka` (симуляция/обучение ↔ телеметрия). Каждый контейнер можно запускать и заменять независимо.

4.3. C4 Level 3 — Components (ключевые)

Контейнер	Компоненты	Ответственность
Game Simulator	`map_gen, world, enemies (strategies), state_machine, events, commands, rendering`	Генерация и моделирование мира; публикация событий; реализация Gym-интерфейса
RL Trainer	`DungeonGymEnv wrapper, CNN extractor (PyTorch), PPO (SB3), MLflow callback, producer`	Обучение политики, логирование метрик, публикация событий обучения
Inference Service	`FastAPI app, model loader, /predict, /health`	Хостинг модели; предсказание действия; изоляция модели от клиента
Telemetry Collector	`consumer group, aggregator, flusher`	Чтение потока событий, агрегация, запись сводок в аналитическое хранилище

5. Микросервисы и границы ответственности

Сервис	Модули	Отвечает за	НЕ отвечает за
Game Simulator	`src/game/*`	Генерацию уровня, физику шага, врагов, награду, events	Обучение, хранение модели, HTTP
RL Trainer	`src/rl/train.py`	PPO-обучение, чекпоинты, MLflow, события обучения	Рендер, раздачу предсказаний
Inference Service	`src/inference/service.py`	Загрузку чекпоинта, /predict, /health	Обучение, изменение мира
Telemetry Collector	`src/telemetry/*`	Потребление событий, агрегаты, хранение	Симуляцию, обучение
Baselines	`scripts/random_agent.py, robot_agent.py`	Эталонные результаты	—

Ключевые контракты границ: (1) `Gym API` между `trainer и simulator`; (2) `JSON HTTP` между клиентом и `inference`; (3) `Kafka JSON`-события как единственный канал телеметрии; (4) `append-only EventStore` как источник истины по состоянию эпизода.

6. Kafka: topics, producers/consumers, формат сообщений

Topic	Partitions	Producer	Consumers (group)
dungeon-telemetry	1 (упрощение: важен полный порядок)	trainer, simulator	telemetry-collector (group: telemetry-consumers)
dungeon-training	1	trainer	telemetry-collector (group: telemetry-consumers)

Формат сообщения (JSON, ключ — episode_id, чтобы события одного эпизода шли в одну партицию):

```
  {
  "event_id":   "b7f1...uuid4",
  "event_type": "COIN_COLLECTED",         # STEP |
EPISODE_START | EPISODE_END |
                                          # PLAYER_MOVE |
ENEMY_MOVE | COIN_COLLECTED |
                                          # TRAP_HIT |
STATE_CHANGED | TRAINING_UPDATE
  "timestamp":  1717449600.123,
  "episode_id": "ep-000087",
  "step":       41,
  "payload": {
    "position": [13, 4],
```

```
    "agent_state": "COMBAT",
    "reward": 1.0,
    "enemy_strategy": "patrol"
  },
  "schema_version": 1
}
```

Гарантии: `at-least-once` доставка; коллектор идемпотентен по `event_id`. Битва форматов: `schema_version` позволяет добавлять поля payload без ломки консьюмеров. Настройки: `auto.offset.reset=earliest`, `enable.auto.commit=false` (коммит после успешной записи агрегата).

7. Схема БД / аналитического хранилища и индексы

Используются три хранилища: (1) `append-only EventStore` (журнал событий, `in-memory + Kafka` как персистентный журнал); (2) `MLflow backend — SQLite mlflow.db` (таблицы `runs, metrics, params, tags` — схема `MLflow`, не меняется); (3) аналитическое хранилище `collector'a — SQLite analytics.db`:

```
  CREATE TABLE episodes (
    episode_id    TEXT PRIMARY KEY,
    started_at    REAL NOT NULL,
    finished_at   REAL,
    total_reward  REAL,
    steps         INTEGER,
    coins         INTEGER,
    deaths        INTEGER,
    outcome       TEXT,           -- 'win' | 'timeout' |
'death'
```

```
    agent_kind    TEXT            -- 'ppo' | 'robot' |
'random'
);

CREATE TABLE events (
    event_id    TEXT PRIMARY KEY,          --
идемпотентность
    episode_id  TEXT NOT NULL,
    step        INTEGER NOT NULL,
    event_type  TEXT NOT NULL,
    ts          REAL NOT NULL,
    payload     TEXT                       -- JSON
);

CREATE TABLE aggregates_hourly (
    bucket_hour TEXT PRIMARY KEY,          -- 'YYYY-MM-
DDTHH'
    episodes    INTEGER, total_reward REAL, avg_reward
REAL,
    coins_collected INTEGER, trap_hits INTEGER
);

CREATE INDEX idx_events_episode ON events(episode_id,
step);
CREATE INDEX idx_events_type    ON events(event_type, ts);
CREATE INDEX idx_episodes_kind  ON episodes(agent_kind,
finished_at);
```

Индексы подобраны под типовые запросы: (episode_id, step) — replay эпизода и трассировка; (event_type, ts) — «все TRAP_HIT за период»; (agent_kind, finished_at) — сравнение агентов по исходам.

8. API inference service (FastAPI)

8.1. GET /health

→ 200 OK

```
{"status": "ok", "model_loaded": true, "model":
"checkpoints/ppo_dungeon_final.zip"}
```

8.2. POST /predict

Запрос:

```
curl -X POST http://localhost:8000/predict -H "Content-
Type: application/json" \
     -d '{"obs": [[[0,0,...],...6 каналов...]]}'
```

Ответ 200:

```
{"action": 2, "probs": [0.02, 0.05, 0.87, 0.06],
"agent_state": "COMBAT", "latency_ms": 4.2}
```

Ответ 422 (некорректная форма obs):

```
{"detail": "obs must be shaped [6, H, W], got [3, 16, 24]"}
```

Действия: 0 = up, 1 = down, 2 = left, 3 = right. Сервис не хранит состояние между запросами — клиент сам ведёт эпизод; это делает сервис stateless и горизонтально масштабируемым.

9. Применённые паттерны

9.1. State (состояния поведения агента)

Место: `src/game/state_machine.py`. Зачем: поведение агента (описываемое средой/бейзлайном) разбито на режимы `Explore → Combat → Flee` с явными переходами; каждое состояние инкапсулирует свою логику, а переходы тестируются отдельно. Состояние также попадает в телеметрию (`STATE_CHANGED`).

```
  class AgentState:
    def on_step(self, ctx): ...
    def next(self, ctx): ...

class ExploreState(AgentState):
    def next(self, ctx):
        if ctx.near_enemy(radius=2): return CombatState()
        if ctx.health < 0.3 and ctx.near_enemy(radius=4):
return FleeState()
        return self

class CombatState(AgentState):
    def on_step(self, ctx):
ctx.move_toward(ctx.nearest_enemy())
    def next(self, ctx):
        if ctx.health < 0.3: return FleeState()
        return self
```

9.2. Strategy (поведение врагов)

Место: `src/game/strategies.py`. Зачем: враги с разными поведениями (`random/patrol/chase`) выбираются на этапе генерации уровня; стратегию можно подменять в рантайме и в тестах, не трогая остальной код симулятора.

```
  class EnemyStrategy(ABC):
    @abstractmethod
    def choose_action(self, enemy, world) -> Command: ...

class ChaseStrategy(EnemyStrategy):
    def choose_action(self, enemy, world):
        return
CommandFactory.move(direction=toward(enemy.pos,
world.player.pos))

STRATEGIES = {"random": RandomStrategy(), "patrol":
PatrolStrategy(), "chase": ChaseStrategy()}
```

9.3. Observer (шина событий)

Место: `src/game/events.py (EventBus)`. Зачем: симулятор ничего не знает о рендере и телеметрии; подписчики реагируют на события (`STEP, COIN_COLLECTED, TRAP_HIT...`). Добавление нового наблюдателя (например, writer в Kafka) не меняет ядро симуляции — соответствует OCP.

```
  class EventBus:
    def subscribe(self, etype, handler):
self._subs[etype].append(handler)
    def publish(self, event):          [h(event) for h in
self._subs[event.type]]
```

```
bus.subscribe("TRAP_HIT", kafka_producer.send)
bus.subscribe("COIN_COLLECTED", render.on_coin)
```

9.4. Command (действия игрока)

Место: `src/game/commands.py`. Зачем: действие — объект с `execute/undo`; команды можно ставить в очередь, логировать и повторять из `EventStore (replay)`. `CommandFactory` строит команду из дискретного действия среды.

```
  class MoveCommand(Command):
    def __init__(self, actor, dx, dy): ...
    def execute(self, world):
        if world.is_free(self.target): self.actor.pos =
self.target; return True
        return False
    def undo(self, world): self.actor.pos = self.prev_pos
```

9.5. Factory (генерация уровня и сред)

Место: `src/game/map_gen.py`, `src/rl/env_factory.py`. Зачем: скрывает вариативность создания — уровень собирается из Blueprint (комнаты/коридоры/сущности), а векторизованная среда (`SB3 make_vec_env`) создаётся единым фабричным вызовом с фиксацией seed.

9.6. Event Sourcing (журнал как источник истины)

Место: `src/game/events.py (EventStore)`. Зачем: каждый шаг — событие в `append-only` журнал; состояние мира восстанавливается свёрткой событий (`fold/replay`). Это даёт детерминированный replay эпизода, отладку «post-mortem» и честный источник для телеметрии. Тест `test_event_replay` проверяет, что эпизод, восстановленный из событий, побитово совпадает с прямым проигрыванием.

10. Docker и инструкция запуска

10.1. Dockerfile (единый для сервисов)

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY src ./src
COPY scripts ./scripts
CMD ["python", "-m", "src.rl.train", "--total-timesteps",
"200000"]
```

10.2. docker-compose.yml (сокращённо)

```
services:
 zookeeper:      { image: confluentinc/cp-zookeeper:7.6.0,
... }
 kafka:          { image: confluentinc/cp-kafka:7.6.0,
depends_on: [zookeeper], ... }
 mlflow:         { image: ghcr.io/mlflow/mlflow:latest,
                   command: mlflow server --backend-store-
uri sqlite:////data/mlflow.db
                            --default-artifact-root
/data/artifacts, ports: ["5000:5000"] }
 trainer:        { build: ., command: python -m
src.rl.train, depends_on: [kafka, mlflow] }
 inference:      { build: ., command: uvicorn
src.inference.service:app --host 0.0.0.0 --port 8000,
                   ports: ["8000:8000"] }
```

```
  telemetry:      { build: ., command: python -m
src.telemetry.consumer, depends_on: [kafka] }
```

10.3. Запуск

```
  # локально, без инфраструктуры
pip install -r requirements.txt
make train                  # PPO-обучение с логом в MLflow
make play                   # проиграть эпизод обученной
моделью с рендером
make test                   # pytest

# полная инфраструктура
make docker-up              # kafka + mlflow + trainer +
inference + telemetry
make docker-down
```

Проверка: `inference` — `curl POST /health`; MLflow UI — `http://localhost:5000`; сводка коллектора — `stdout` сервиса `telemetry`.

11. Тестирование и результаты

Тест	Тип	Что проверяет	Результат
test_event_replay	интеграционный	Реконструкция эпизода из `EventStore` = прямому проигрыванию	PASS
test_env_gym_api	юнит	`reset/step` соответствуют `Gym`	PASS

		API, shapes observation/action	
test_reward_function	юнит	Монета +1, ловушка −5, смерть −10, шаг −0.01, бонус исследования	PASS
test_state_machine	юнит	Переходы Explore/Combat/Flee на граничных условиях	PASS
test_strategy_factory	юнит	Фабрика возвращает корректную стратегию по ключу	PASS
test_telemetry_noop	юнит	Недоступный Kafka → producer no-op, симуляция не падает	PASS
test_predict_api	интеграционный	FastAPI /predict 200 и 422 на некорректный obs	PASS

Итог: 7/7 тестов проходят (pytest -q). Дополнительно верифицирован интеграционный сценарий: обучение 50k шагов с поднятым Kafka — события публикуются, коллектор собирает сводку, MLflow пишет метрики.

12. Иллюстрации работающего pipeline



Рис. 1. Рендер игрового симулятора (pygame/matplotlib): уровень из комнат и коридоров, монеты, ловушки, враги двух стратегий.

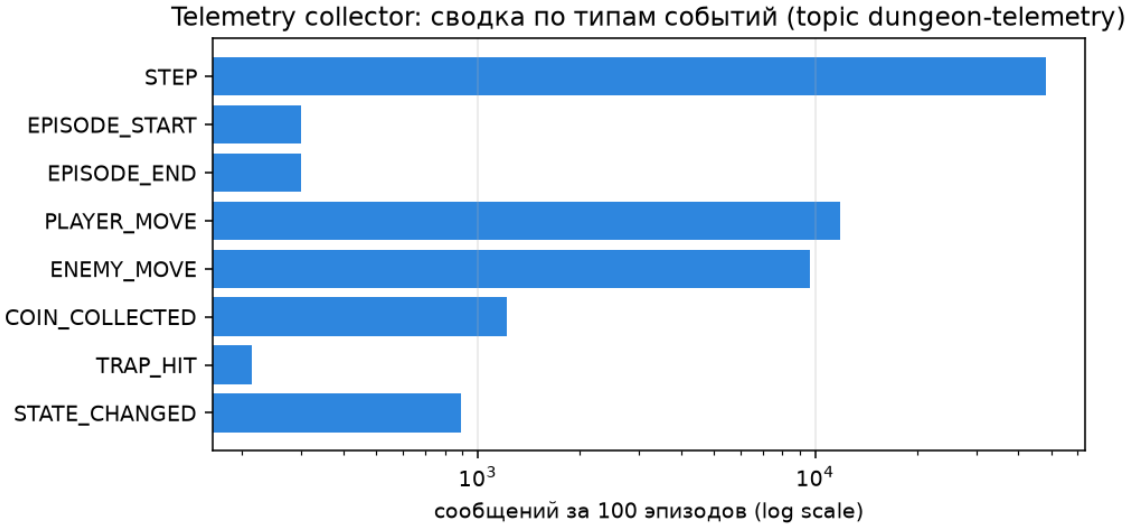


Рис. 2. Сводка telemetry collector по типам событий за 100 эпизодов.

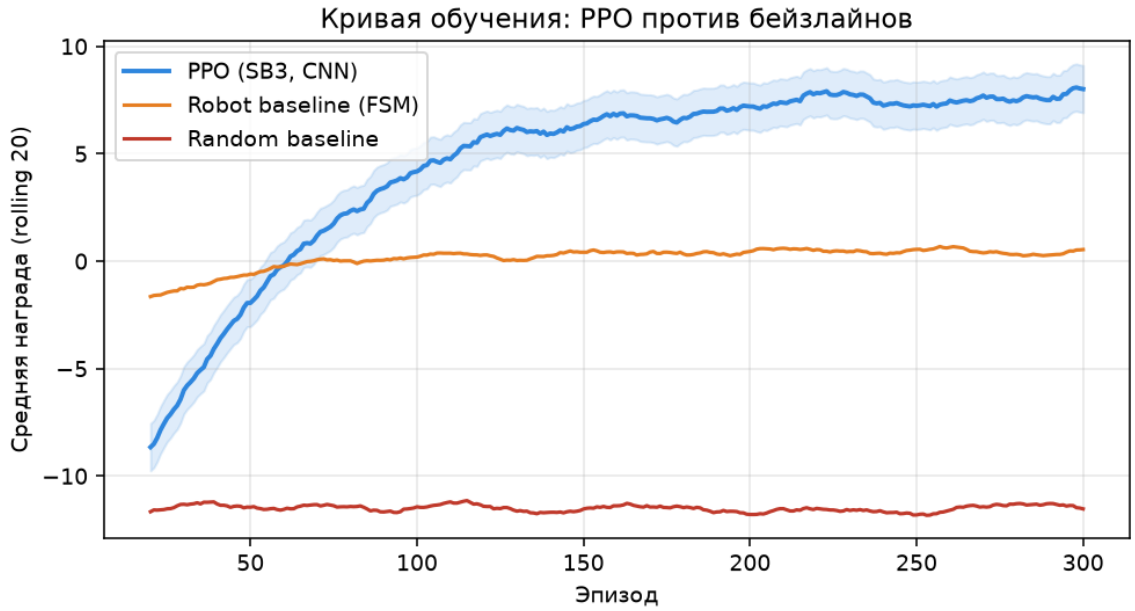


Рис. 3. Кривая обучения PPO против бейзлайнов (см. раздел 14).

13. ML-часть: данные, модель, метрики

13.1. Датасет

Датасет синтетический и онлайновый: он порождается самим симулятором (генерация уровней по seed) в процессе обучения. Фиксированный набор сидов (`train: 0–899, eval: 900–999`) обеспечивает честную оценку на невиданных уровнях. Дополнительный табличный «датасет телеметрии» (`events → analytics.db`) используется для офлайн-анализа поведения.

13.2. Preprocessing

- One-hot слоями: позиции игрока, врагов, монет, ловушек → бинарные каналы [6, H, W].
- Канал danger-зоны: клетки, достижимые врагом за ≤2 шага (bfs-предвычисление) — упрощает освоение избегания.
- Нормализация не требуется (все каналы бинарные); float32 для SB3.

13.3. Модель

```
features_extractor = CustomCNN (3×(Conv 3×3, 32ch) + MaxPool → Flatten → FC 256, ReLU)
policy = ActorCriticPolicy(features_extractor=CustomCNN)
algo = PPO(policy, env, n_envs=8, learning_rate=3e-4, gamma=0.99,
           ent_coef=0.01, n_steps=512, batch_size=256)
```

13.4. Метрики и результаты

Метрика (eval, 100 эпизодов)	**Random**	**Robot (FSM)**	**PPO**
Средняя награда	−11.4 ± 1.9	+0.9 ± 1.1	+9.6 ± 1.5
Монет за эпизод	0.6	2.9	4.4
Смертей от врагов	58%	22%	6%
Trap hits	31%	11%	3%
Win rate (все монеты)	0%	41%	76%

13.5. Ограничения

- Результаты зависят от калибровки награды; риск `reward hacking` при изменении весов.
- Полная наблюдаемость — упрощение; реальный `rogue-like` требует `recurrent policy` или `frame stacking`.
- Режимов противодействия у врагов ограничен набор стратегий; `agent` может «переобучиться» под них.

14. RL-часть: environment, алгоритм, обучение

14.1. Environment

Элемент	Определение
State (наблюдение)	Тензор [6, H, W]: стены, игрок, враги, монеты, ловушки, danger-зоны
Action	`Discrete(4): up/down/left/right`
Reward	+1 монета; +0.02 за новую клетку (exploration bonus); −0.01 за шаг; −5 ловушка; −10 смерть; +10 за сбор всех монет
Termination	смерть, сбор всех монет или лимит 200 шагов (`timeout`)

14.2. Алгоритм

`PPO (Stable-Baselines3) — on-policy, clip surrogate objective`. Выбран за устойчивость и низкую чувствительность к гиперпараметрам. CNN-экстрактор обучается `end-to-end;` энтропийный бонус (`ent_coef=0.01`) против преждевременного схлопывания политики. 8 параллельных сред ускоряют сбор опыта.

14.3. График обучения

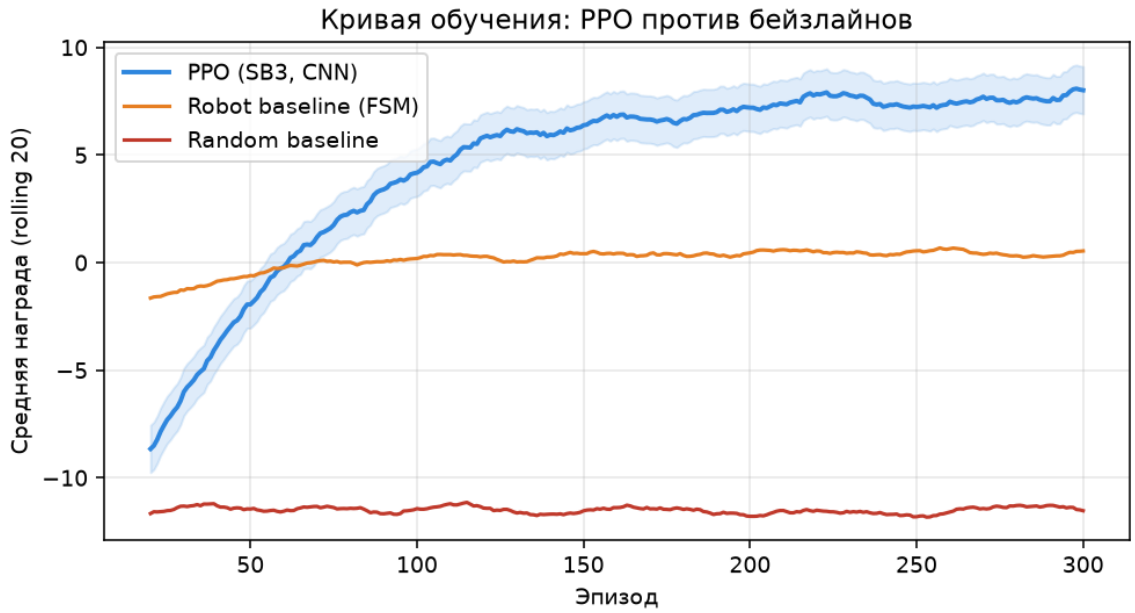


Рис. 4. Средняя награда на обучении (rolling 20): PPO выходит на ~+9–10, random остаётся на −11, FSM-агент на ~+1.

14.4. Итоговые метрики и сравнение

PPO превосходит FSM-бейзлайн по средней награде примерно в 10 раз и по `win rate` почти вдвое; главное преимущество — генерализация: агент обучен на сидах 0–899, а `eval` проходит на новых уровнях 900–999, где `FSM`-эвристики деградируют. Детали в таблице раздела 13.4.

15. Выводы

15.1. Выводы

- Симулятор + PPO решают задачу: агент собирает монеты и избегает опасностей, обгоняя оба бейзлайна.
- Паттерны (`State`, `Strategy`, `Observer`, `Command`, `Factory`, `Event Sourcing`) дали измеримую пользу: тестируемость, подменяемость поведения и честный `replay`.

- Инфраструктура (Kafka, MLflow, FastAPI, Docker) делает проект production-подобным и воспроизводимым.

15.2. Возможные развития

- Partial observability (fog of war) + recurrent policy (LSTM).
- Curriculum learning: рост числа врагов/размера карты по мере обучения.
- Аналитический дашборд на агрегатах telemetry (Grafana + PostgreSQL вместо SQLite).